

Agent教程

📅 2026年，Agent（智能体）已从概念走向大规模落地。下文将从核心概念到实战开发，提供一份完整的Agent学习指南。

一、什么是Agent？——从“聊天机器人”到“数字员工”

1.1 核心定义

Agent（智能体）是一种能够自主感知环境、制定计划、调用工具、执行行动，并根据反馈动态调整策略的AI系统。

智能体核心组件架构



1.2 思维链

大模型思维链（Chain of Thought，简称 CoT）是一种提示策略，旨在引导大型语言模型（如 GPT-5、DeepSeek、Claude 等）在给出最终答案之前，先输出一系列中间推理步骤。

简单说，就是让模型像人一样“一步步想，再回答”。

1.3 快速直觉 & 慢速推理

卡尼曼的"系统1"（快速直觉）和"系统2"（慢速推理）理论正在深刻改变Agent的设计范式。传统的Agent要么完全依赖快速但可能不够准确的直觉判断，要么对所有问题都进行深度推理导致效率低下。而受双系统理论启发的新一代架构，通过"快慢结合"实现了效率与智能的平衡。



以下是当前该理论在Agent领域的主要应用方向：

💡 核心应用架构：分离"对话"与"推理"

这是目前最主流的应用模式，将Agent明确拆分为两个协同工作的子系统：

系统	类比	核心职责	技术实现
System 1 (快系统)	"Talker"或"行为系统"	负责快速、直觉的响应，如生成日常对话、维持流畅的交互、做出本能反应。	通常由一个轻量级或经过专门微调的语言模型驱动，能够毫秒级地生成回复。
System 2 (慢系统)	"Reasoner"或"认知系统"	负责复杂的逻辑推理、多步规划、调用外部工具（如搜索引擎、计算器）、进行深度分析和决策。	基于更强大的语言模型（如GPT、DeepSeek），并整合了思维链、外部知识库和工具调用能力。

这种分工使得Agent既能与人自然闲聊，又能解决复杂的数学问题。

1.4 模型选型

为大模型选择作为其"大脑"的底座模型，是Agent开发中决定成败的战略决策。选型的本质，是为你的特定任务找到性能、成本与安全之间的最佳平衡点，而非寻找一个通用领域的“最强王者”。

以下是从五个核心维度构建的选型框架。

核心能力：让Agent学会"思考与行动"

Agent所需的核心能力，远比普通的对话机器人要复杂，主要包括：

- **复杂推理与规划 (Reasoning & Planning):** 准确理解用户目标并将其拆解为可执行步骤的能力。这需要模型具备强大的思维链 (Chain-of-Thought) 能力。
- **工具调用 (Function/Tool Calling):** 根据规划，准确调用外部工具（如API、搜索引擎、代码解释器）并正确填充参数的能力。
- **长上下文处理 (Long-Context):** 有效处理和记忆长对话历史、大型文档的能力，这对需要深入分析的Agent至关重要。

主流模型性能速览（截至2025年底）：

- **Agent与工具调用：** Claude系列（Sonnet 4/Opus 4.1）与智谱的GLM-4.5在BFCL上得分超过70%，处于领先水平。
- **软件工程：** Claude Opus 4.5在SWE-Bench测试中得分高达80.9%，GPT-5.2 (Codex)紧随其后达80%，两者均为顶尖的编码Agent模型。
- **自主操作：** Claude 4.5在OSWorld（桌面/网页任务）基准上得分**61.4%**，表现出更强的计算机使用能力。

成本与效率：找到性价比的"甜点区"

成本是决定Agent项目能否规模化落地的关键因素，主要包含：

- **API调用费用：** 闭源模型按Tokens数收费，Agent一次复杂任务可能涉及多次模型调用，成本需精确估算。
- **部署与运维成本：** 自部署开源模型（如Llama 3、Qwen 2.5）需要考虑GPU服务器的硬件、维护和电力成本。
- **延迟 (Latency):** 模型的响应速度直接影响用户体验。对于需要实时交互的场景，更小、更快的模型（如GPT-4o, Gemini 2.5 Flash）可能是更好的选择。

⚠️ 避坑指南

避免一开始就过度追求顶级模型的“完美”表现。一个“小模型为主，大模型兜底”的混合方案，如使用Qwen 2.5-7B等开源模型处理日常任务，仅在遇到复杂情况时调用GPT-5等顶级模型，往往是平衡成本与性能的更优策略。

🔒 安全与合规：保障企业应用的基石

对于企业级应用，安全合规是必须通过的“高压线”。

- **数据隐私：**明确数据是否会被用于模型训练。金融、医疗等敏感领域应优先考虑支持私有化部署的开源模型（如Qwen、DeepSeek、Llama）。
- **合规要求：**不同行业有不同法规（如GDPR、HIPAA），需选择符合相应标准的模型和服务商。
- **部署方式：**评估模型供应商是否提供公有云、私有云、混合云等多种灵活的部署选项。

🚀 未来考量：构建可演进的架构

大模型技术迭代极快，选型时需考虑长远：

- **模型中立性 (Model Agnostic)：**架构上应避免与单一模型深度绑定。优秀的设计应能方便地替换底层模型，以快速跟上技术发展。
- **标准化协议：**优先选择支持MCP（模型上下文协议）等开放标准的模型和工具，这将极大增强系统的可扩展性和工具生态的兼容性。

💡 选型速查

总的来说，没有“最好”的模型，只有“最合适”的模型。以下是分场景的速查建议：

核心场景	首选模型	选型理由
👤 复杂编码与系统调试	Claude Opus 4.5	拥有顶尖的SWE-Bench得分（80.9%）和强大的计算机使用能力（OSWorld 61.4%），是完成复杂工程任务的利器。
💡 高性价比Agent开发	开源模型 (如 Qwen 2.5, Llama 3)	通过SFT微调适配垂直领域，成本可控。可构建“小模型为主，大模型兜底”的智能路由架构，平衡成本与性能。
⚡ 实时语音/多模态交互	GPT-4o / Gemini 2.5 Flash	专为低延迟、多模态交互设计，适合需要实时响应的应用场景。

核心场景	首选模型	选型理由
 长文档分析与研究	GPT-4.1 / Gemini 2.5 Pro	两者均支持高达1M-2M tokens的超长上下文，非常适合处理大型文档和研究报告。
 高安全合规企业场景	开源/国产模型 (如 文心一言, Qwen)	支持私有化部署或在国内有合规的服务节点，能够满足金融、政务等行业严格的数据安全要求。

1.5 应对模型幻觉

“模型幻觉”指大语言模型生成的内容看似逻辑通顺、言之凿凿，但实际上与事实不符、凭空捏造或无可靠信源支撑的现象。

它通常表现为：

- 虚构细节：编造不存在的书名、论文标题、人物履历、法律条文编号。
- 合成事实：把两件真实但无关的事强行拼接成逻辑闭环。
- 过度自信：即便不知道答案，也会用非常肯定的语气给出错误信息。

核心成因

幻觉不是程序出错，而是大模型基于概率预测下一个词的工作方式的必然产物。当模型遇到知识盲区，它不会“说不知道”，而是根据语言模式猜测一个“看起来最像正确答案”的表述。

应对与缓解建议

对于Agent开发者而言，幻觉不再仅仅是一个“回答错误”的问题，而是会导致整个自动化流水线崩溃、执行非法操作或产生严重级联错误的系统性风险（例如：编造不存在的API参数、伪造工具执行结果、在长程规划中遗忘目标等）。

应对Agent的幻觉，不能仅靠Prompt说“请不要产生幻觉”，而必须从架构设计、工具约束、状态管理和反馈循环四个维度进行系统性缓解。以下是针对Agent开发者的实战建议：

一、工具与执行层约束

Agent幻觉最致命的区域发生在“决定调用什么工具”和“如何传递参数”时。

1. 严格的JSON Schema与强类型校验

- 问题：Agent经常发明不存在的工具名，或传入错误格式的参数（如把字

字符串传给需要整数的字段）。

- 缓解：在代码侧（而非模型侧）定义严格的Pydantic模型或JSON Schema。如果LLM输出的工具调用无法被解析，直接抛出异常并重试，绝不进行“模糊匹配”或“猜意思”。

2. 工具描述的“防幻觉”设计

- 问题：描述模糊会导致LLM“脑补”工具能力。
- 缓解：工具描述必须遵循“穷举边界”原则。明确写出：能做什么、不能做什么、参数的精确含义、返回值的精确结构。

3. 引入“守门员”模式

- 对于高风险操作（如删除数据、发邮件、执行交易），不要让规划模型的输出直接触发动作。引入一个轻量级模型（如被严格Prompt约束的小模型）专门负责审核工具调用的合法性。

二、规划与推理层增强

Agent在多步推理中容易产生“逻辑幻觉”（如步骤跳跃、伪造中间结果）。

1. 强制分离“思考”与“行动”

- 严格遵循ReAct模式。在LLM输出Action之前，强制要求输出Thought。
- 在Thought阶段，Prompt中必须要求模型引用上下文中的原文或前一步的精确Observation，禁止使用模糊的主观推测（例如要求：“在Thought中必须包含引用证据：[...]”）。

2. 显式错误传播与纠正

- 问题：工具执行报错后，Agent有时会假装成功，并基于虚假结果继续规划。
- 缓解：当工具返回错误时，在Prompt中用极度显眼的标识（如[SYSTEM ERROR: ...]）包裹，并在System Prompt中强硬规定：“如果你看到[SYSTEM ERROR]，你必须承认上一步失败，并尝试其他方案，绝不允许假设操作成功。”

3. 自我反思与批判者注入

- 在长链路任务中，每隔N步插入一个“Reviewer”角色（可以是同一个LLM的不同对话轮次，也可以是另一个专门挑错的LLM）。
- 提示词示例：“请审查上述执行轨迹。找出其中逻辑不连贯、与初始目标偏离、或缺乏证据支撑的步骤。如果没有，输出PASS；如果有，指出具体错误。”

4. 树搜索与多路径探索

- 放弃单链路的ReAct，采用Tree of Thoughts (ToT) 或 LATS (Language Agent Tree Search)。让Agent探索多条路径，通过外部评估函数（如代码执行结果、规则匹配）来剪枝幻觉分支，而不是盲信LLM的某一次输出。

三、记忆与上下文层管理

长文本下的遗忘和上下文污染是Agent幻觉的重灾区。

1. “代码即状态机”原则

- 核心法则：永远不要让LLM维护全局状态！
- 不要问LLM“我们刚才执行到哪一步了？”，而是由你的Python代码维护一个状态字典，每次将当前状态直接注入Prompt：“当前状态: {step: 3, target_file: 'main.py', error_count: 1}”。

2. 警惕“摘要幻觉”

- 当对话历史过长需要压缩时，LLM生成的摘要往往会丢失关键细节或无中生有。
- 缓解：采用滑动窗口 + 结构化提取代替自由文本摘要。用代码提取关键实体（如涉及的文件名、报错信息、已完成的Action列表），以JSON格式存入长期记忆，而非让LLM写一段总结。

3. RAG检索质量的兜底

- Agent在调用知识库时，如果没有检索到相关内容，很容易基于自己的预训练知识“强行回答”。
- 缓解：在RAG的Prompt中加入硬性指令：“如果提供的References中没有包含回答该问题所需的信息，你必须回答‘根据已知信息无法回答’，禁止使用你的内置知识。”

四、架构与后处理层兜底

将信任边界从LLM转移到确定性代码上。

1. 断言与输出校验

- 对于Agent的最终输出或中间关键数据提取，必须用正则表达式或代码逻辑进行拦截。
- 例如，Agent提取了10个订单号，用代码检查是否符合 `\d{10}` 的格式，

不符合的直接丢弃并要求重试。

2. 多智能体辩论

- 对于复杂的决策，使用多个具有不同System Prompt（甚至不同底层模型）的Agent进行独立规划，然后通过一个“法官Agent”比对它们的输出。如果发现矛盾点，则打回重做。这在很大程度上能过滤掉单模型的随机性幻觉。

3. 人机协同的优雅降级

- 设计置信度评分机制（可以通过让LLM对自身输出打分，或者通过多次采样计算一致性）。
- 当置信度低于阈值，或触发关键词（如“删除”、“支付”、“不确定”），立即中断Agent循环，将上下文和当前选项推送给人类审批。

💡 Agent开发者的“反幻觉”心智模型

作为开发者，你需要建立一个基本认知：**LLM本质上是一个“带有概率性的文本补全器”，它不具备执行逻辑的能力。**

- ❌ 错误心智：写一段Prompt，把任务丢给LLM，希望它像人一样自己思考、查错、完成。
- ✅ 正确心智：你（开发者）在写代码，**LLM只是你代码里的一个“不确定性的函数”**。你需要用 `if/else`、`try/except`、`Pydantic`、`状态机` 将这个函数的输入输出死死框住。任何不能被代码验证的LLM输出，都应被视为潜在的幻觉。

1.6 提示技巧

和 AI Agent 打交道，跟用传统搜索引擎或普通聊天机器人不太一样。Agent 不仅能回答问题，更能自主规划、调用工具、执行多步任务。想让 Agent 发挥出真正的能力，提示词需要从“指令”升级为“任务简报”。

以下是 7 个核心技巧，按从基础到进阶的顺序展开。

1.6.1 角色设定 + 目标对齐

Agent 需要明确的身份锚点来调动对应的知识库和工具集。

- 基础版：“帮我查资料。”
- Agent 版：“你是一名资深行业分析师。请针对 2025 年 AI 手机市场，梳理出 TOP3 厂商的最新动态，最后用 SWOT 表格呈现。”

1.6.2 明确授予“工具权限”

Agent 区别于普通 LLM 的核心在于它能动手（搜索、写代码、运行脚本）。提示词中若不提醒，Agent 可能会用纯文本“编造”答案。

- 技巧：直接告诉它可以/应该用什么工具。
- 示例：“请使用联网搜索核实以下数据，不要凭记忆回答。如果数据有冲突，列出信源对比。”

1.6.3 任务规划而非一步到位（CoT 思维链）

Agent 擅长长链条任务，但容易跑偏。用强制规划来约束它的思考路径。

- 提示词结构：

“在动手前，请先列出你的执行步骤清单（Plan）。
第一步：... 第二步：...
确认规划无误后，再开始执行第一步。”

1.6.4 极端明确的约束条件

Agent 自主性高，若不设边界，它会疯狂调用工具浪费 Token 或陷入死循环。

- 必须加上的约束：
 - 搜索深度：“最多搜索 3 个页面即可停止，直接总结。”
 - 输出篇幅：“最终报告不超过 500 字，用 3 个要点概括。”
 - 终止条件：“如果连续两次搜索无果，请直接回复‘暂无确切信息’而非继续尝试。”

1.6.5 结构化输出格式（硬指令）

对于需要后续导入 Excel、代码或数据库的结果，必须用 Markdown 语法规规定死格式。

- 错误：“把数据列出来。”
- 正确：“输出格式要求：仅输出 JSON 代码块，键名为 `company`, `revenue`, `year`。不要包含任何解释性文字。”

1.6.6 人机回环

Agent 执行高风险操作（如发邮件、修改代码库、大额计算）前，强制它向你确认。

• 技巧：

“当你准备好发送邮件草稿时，请先输出完整的邮件正文供我审阅，并询问‘是否确认发送？’。在我回复‘确认’之前，严禁执行发送动作。”

1.6.7 防幻觉“引用铁律”

Agent 搜索能力变强了，但幻觉依然存在。

• 杀手级提示词：

“每一条事实性结论后面，必须附上 [来源：具体URL或文档段落]。如果某条信息是你的推测，请明确标注 [注：基于常识推断]。”

实战模板：一个让 Agent 写周报的对比

低效提示	高效 AGENT 提示
帮我写个本周工作总结。	角色：你是我的一号助理。 任务：基于我在 Slack 频道 #project-x 和 Notion 任务板的记录，撰写本周项目进展周报。 规划要求：1. 先读取对应文档；2. 提取关键里程碑；3. 识别风险项。 输出格式：分三部分：本周产出、下周计划、风险求助。 约束：仅读取最近 7 天的记录，不要访问无关频道。

1.7 Agent vs 大模型 vs AI助手

层级	代表产品	核心特点	局限
大模型	GPT-5、DeepSeek	"超级语言引擎"，给定输入、输出文本	被动响应、无记忆、不会主动行动
AI助手	ChatGPT、豆包	有多轮对话与记忆管理能力	本质仍是"人问、AI答"的被动交互
Agent	CrewAI、定制化智能体	自主感知→规划→行动→反馈→修正的闭环系统	目前仍面临复杂推理与长期记忆的挑战

一句话总结：大模型是"大脑"，AI助手是"会说话的大脑"，而Agent是一个"会行动、会协作、会学习的数字员工"。

二、Agent的工作原理

2.1 PEAS 模型

PEAS 模型是用于定义和描述Agent运行环境与目标的核心框架。它由四个英文单词的首字母组成：



简单理解：**PEAS** 回答了智能体“要达成什么目标、在什么环境里、用什么方式行动、靠什么获取信息”这四个基本问题。

经典例子对比

智能体类型	性能度量 (P)	环境 (E)	执行器 (A)	传感器 (S)
医疗诊断系统	诊断准确率、召回率、解释清晰度	患者症状、检验报告、病史库	显示诊断、开检验单、打印处方	键盘输入症状、读取电子病历 API
垃圾邮件过滤器	正确率（拦截垃圾邮件，不误拦正常邮件）	邮件服务器、发件人、用户反馈	标记邮件、移至垃圾箱、放行	读取邮件文本、解析元数据

2.2 四大核心组件

Google在其Agent白皮书中定义了构成Agent的四大核心组件¹：



- **模型（Model）**：负责推理、规划与决策的核心，不同任务可能需要不同规模的模型来平衡成本与效能。
- **工具（Tools）**：让Agent与真实世界互动的接口，包括搜索网络、读取数据库、执行代码或操作API。
- **协调层（Orchestration）**：连接大脑与双手的关键，负责管理记忆、规划步骤（推理→行动→观察的循环），并确保任务在轨道上运行。
- **部署基础设施（Deployment Infrastructure）**：Agent运行的基础环境，涉及安全性验证、权限管理与可扩展性。

2.3 智能体的"感知→规划→行动→反馈"闭环

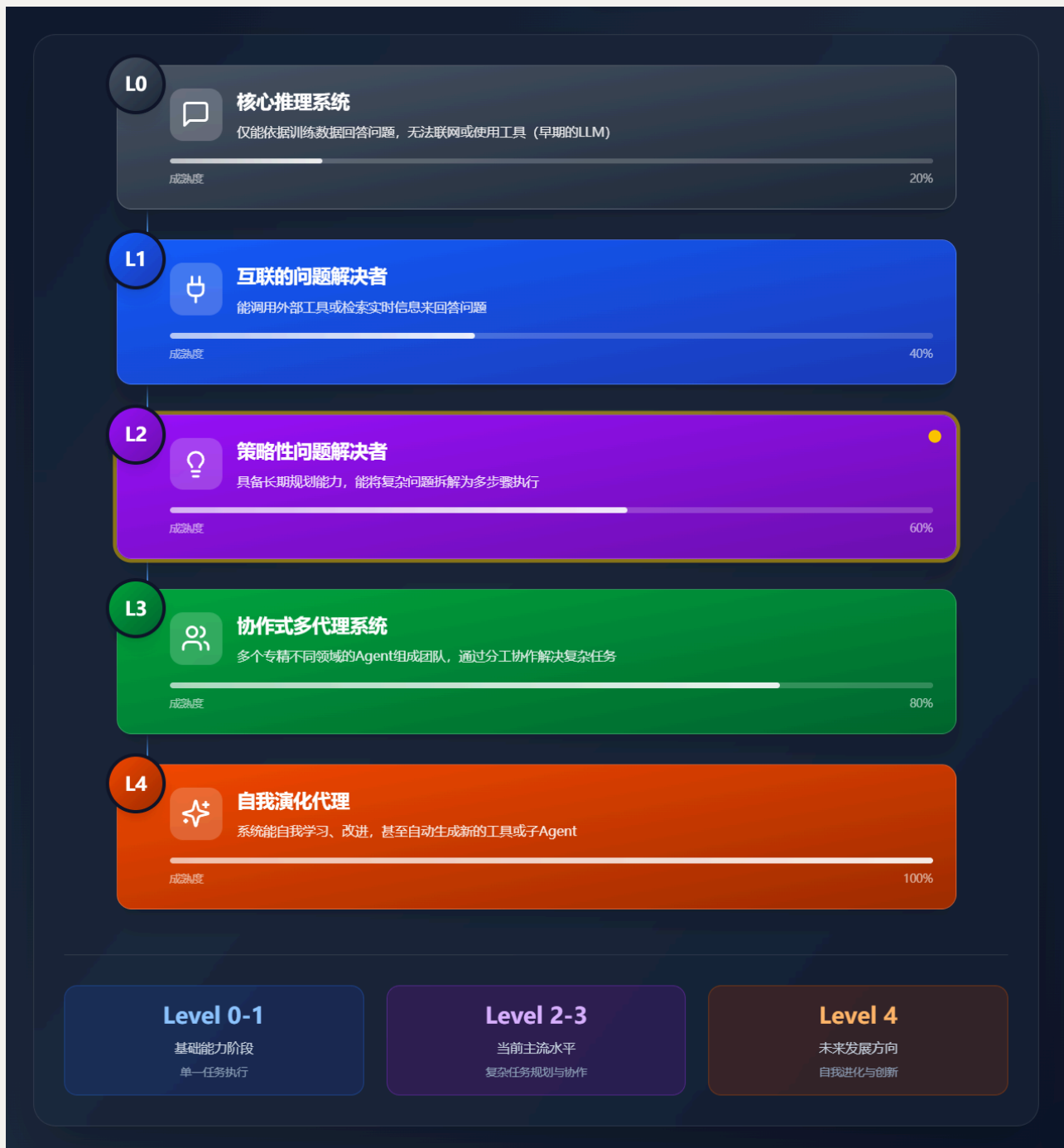
一个完整的Agent工作流程如下：



1. 感知（**Perception**）：接收用户输入和环境信息（文本、图像、API数据等）
2. 规划（**Planning**）：将复杂目标拆解为可执行的子任务序列（使用Chain of Thought等技术）
3. 行动（**Action**）：调用工具执行具体操作（搜索、计算、调用API、写代码等）
4. 反馈（**Feedback**）：收集执行结果，更新记忆状态，动态调整后续策略

2.4 Google的Agent五级进化论

Google将AI Agent分为五个等级，帮助评估当前技术成熟度：



- **Level 0 - 核心推理系统**：仅能依据训练数据回答问题，无法联网或使用工具（早期的LLM）
- **Level 1 - 互联的问题解决者**：能调用外部工具或检索实时信息来回答问题
- **Level 2 - 策略性问题解决者**：具备长期规划能力，能将复杂问题拆解为多步骤执行
- **Level 3 - 协作式多代理系统**：多个专精不同领域的Agent组成团队，通过分工协作解决复杂任务
- **Level 4 - 自我演化代理**：系统能自我学习、改进，甚至自动生成新的工具或子Agent

目前大多数领先企业正处于从Level 2迈向Level 3的阶段。

三、主流Agent开发框架

3.1 技术栈概览

Agent 技术栈已演变为一个包含 模型、框架、记忆、工具、基础设施 等核心层次的复杂系统，旨在让 AI 不仅能“听懂指令”，更能“自主行动”，完成从规划、执行到反思的完整闭环。



📁 向量数据库

- Pinecone
托管向量数据库服务
- Weaviate
开源向量搜索引擎
- Milvus
高性能向量数据库
- Chroma
AI原生的嵌入数据库
- Qdrant
向量相似度搜索引擎
- FAISS
Facebook的相似性搜索库

6 项技术

🔗 工具集成

- LangChain Tools
Agent工具生态系统
- Zapier
自动化工作流集成
- Web Search APIs
Serper, SerpAPI等
- Code Execution
代码解释器和沙箱
- File Processing
PDF, CSV, Excel处理
- API Integration
REST/GraphQL连接器

6 项技术



编排与 workflow

- **LangGraph**
构建状态机 workflow

- **Semantic Kernel**
Microsoft 的 AI 编排框架

- **Haystack**
NLP 流水线框架

- **Flowise**
可视化 LLM 流程构建器

- **Langflow**
拖拽式 LLM 应用构建

- **n8n**
workflow 自动化平台

6 项技术



记忆系统

- **Short-term Memory**
对话上下文管理

- **Long-term Memory**
持久化知识存储

- **Vector Memory**
语义记忆检索

- **Redis**
高速缓存存储

- **PostgreSQL**
结构化数据存储

- **MongoDB**
NoSQL 文档数据库

6 项技术

推理引擎

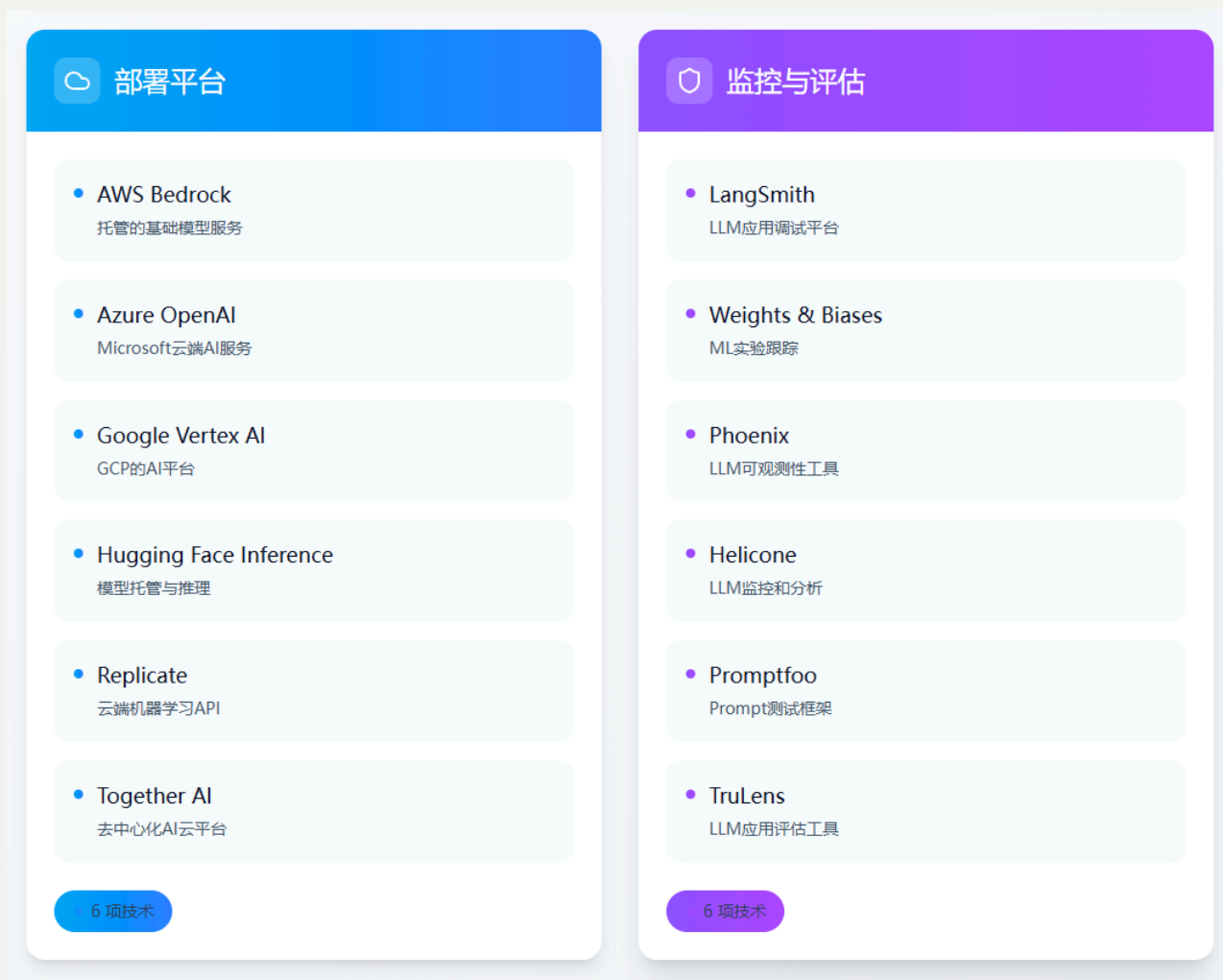
- vLLM
高性能LLM推理服务
- Text Generation Inference
HuggingFace推理服务器
- TensorRT-LLM
NVIDIA优化的推理引擎
- Ollama
本地运行开源模型
- LocalAI
自托管的OpenAI替代
- LM Studio
桌面LLM运行环境

6 项技术

嵌入模型

- OpenAI Embeddings
text-embedding-ada-002
- Cohere Embed
多语言嵌入模型
- Sentence Transformers
开源语义编码器
- BGE Models
BAAI的通用嵌入模型
- M3E
Moka中文嵌入模型
- GTE
阿里达摩院嵌入模型

6 项技术



3.2 低代码平台

平台一：Coze

扣子编程

创建智能体提示词

打造一款高性能的“每日AI简报”智能体。系统支持跨平台数据自动化采集，覆盖36氪、虎嗅、IT之家、InfoQ、GitHub及arXiv等多元渠道，实时追踪AI领域的产业头条、学术前沿与开源生态。通过深度处理与结构化重组，最终输出具备专业深度且高度精炼的常态化行业日报。

低代码平台

- 通过 workflow 创建

平台二：Dify

- 构建全能个人助手：日常助手、文案优化、集成高德地图MCP工具

3.2 框架对比一览

框架	开发商	特点	适用场景
LangChain	开源社区	最成熟的Agent开发生态，丰富的工具链与记忆模块	单Agent开发、RAG应用、初学者入门
LangGraph	LangChain生态	基于图结构的Agent编排，灵活的状态管理与流程控制	复杂 workflows、多步骤Agent
LlamaIndex	LlamaIndex Inc.	能让你的私有或领域数据轻松地与大语言模型结合	构建智能问答、知识库系统等应用
AutoGen	Microsoft	专为多Agent协作设计，支持对话式Agent与人工介入	多Agent系统、研究实验
CrewAI	开源社区	角色驱动的多Agent协作，类似团队分工模式	复杂业务流程自动化

3.3 如何选择适合的框架？

你的情况	推荐框架
Python开发者，想快速上手	LangChain
需要复杂的多Agent协作	AutoGen / CrewAI
不想写代码，低代码平台	Dify / Coze（扣子）

四、Agent推理模式

Agent的推理模式，核心是让AI不仅能“思考”，更能通过“行动”与真实世界互动，来解决复杂问题。这就像是从一个只能纸上谈兵的顾问，升级为能在实战中不断尝试、学习和调整的专家。

目前主流的Agent推理模式，可以根据其复杂度和能力，分为几个层面来理解。

4.1 Agentic推理的三个能力层级

Agentic推理的能力是逐步进阶的，可以划分为三个层级：

1. **基础Agentic推理**：在相对稳定的环境中，能自主完成“规划-行动”的闭环。具备将大任务分解为小步骤（规划）、调用外部工具如搜索引擎或计算器（工具使用），以及主动检索信息（搜索）的能力。例如，让Agent自动编写代码并运行调试，或通过搜索整理一份实时报告。
2. **自进化Agentic推理**：当环境变得不确定时，Agent能通过反馈和记忆从经验中学习，不断自我优化。例如，它可以反思自己的错误（自我反思），或通过强化学习来优化长期策略。这使得Agent能持续进步，而无需每次从头训练。
3. **集体多智能体推理**：多个Agent通过明确的角色分工与协作，形成一个解决问题的能力远超个体的“智囊团”。例如，一个Agent扮演“项目经理”负责任务分解，其他Agent分别负责编码、测试和资料检索，共同完成一个复杂的软件开发项目。

4.2 单Agent的三种核心推理范式

在一个Agent内部，主要采用以下三种推理范式来完成任務：

- **ReAct (Reasoning + Acting)**：将推理与行动交错进行，形成一个“思考→行动→观察→再思考”的循环。Agent每一步都会根据当前观察结果动态决策，灵活调整计划，非常适合步骤不确定、需要灵活应对的复杂任务。其决策过程透明，易于调试。
- **Plan-and-Execute (规划-执行)**：先制定一个完整的执行计划（任务清单），再严格按照计划逐步执行。如果执行中遇到问题，可以重新规划（Replan）。这种模式适合任务流程长、子任务间依赖关系明确的场景。
- **Reflection Loop (自我反思循环)**：在得到结果后，加入一个质量评估环节。如果结果不达标，Agent会根据评估反馈重新生成，直到产出高质量结果或达到重试上限。这通常作为前两种模式的补充，用于对输出质量有严格要求的场景。

4.3 其他重要的推理技术与模式

除了上述核心范式，还有一些重要的技术和模式：

- **纯推理模式**：这类模式专注于“思考”本身，不涉及与外部环境交互。
 - **思维链 (Chain-of-Thought, CoT)**：通过展示一步步的推理过程来解决问题。
 - **思维树 (Tree of Thoughts, ToT)**：探索多条推理路径，并在必要时回溯，适合需要深思熟虑的复杂问题。
 - **思维图 (Graph of Thoughts, GoT)**：允许推理链进行合并、循环等更复杂的操作，形成一个推理网络。

- **多Agent协作模式**：当任务过于复杂时，会采用多个Agent协作的模式。
 - **Orchestrator-Worker (编排-工作者)**：由一个中央“编排者”Agent负责任务分解和分配，多个“工作者”Agent执行子任务。
 - **Supervisor (监督者)**：由一个“监督者”Agent监控和指导其他“子Agent”的工作，确保输出质量。
- **其他重要概念**：
 - **ReWOO (Reasoning Without Observation)**：一种先规划好所有行动再一次性执行的模式，与ReAct的逐步执行形成对比。
 - **启发式 (Heuristics)**：使用经验法则或“捷径”来快速做出决策，提高效率。
 - **条件逻辑 (Conditional Logic)**：基于预定义的“如果-那么”规则进行决策，是最简单直接的方式。

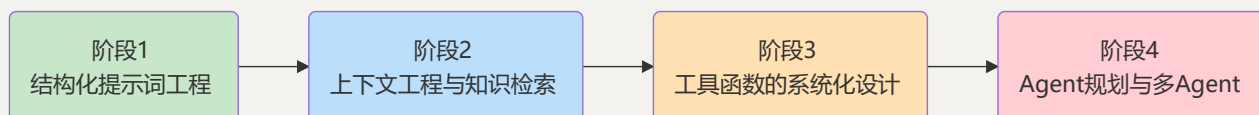
Agent的推理模式是一个从简单到复杂、从个体到集体的完整光谱。你可以根据任务的特性来选择合适的模式：

- **简单、规则明确的任务**：可直接使用条件逻辑或启发式方法。
- **需要多步推理但无需外部交互**：**CoT**、**ToT**等纯推理模式是很好的选择。
- **需要与外部世界动态交互**：**ReAct**模式提供了强大的灵活性。
- **任务流程清晰、步骤固定**：**Plan-and-Execute**模式更为高效。
- **对结果质量要求极高**：务必引入**Reflection Loop**进行质量把控。
- **任务极度复杂**：可以考虑设计多Agent协作系统。

五、Agent学习路径（从零到一）

5.1 四阶段学习路径

Agent的学习可以分为四个阶段²：



阶段1：结构化提示词工程

- 学习设计高效、可复用的提示词模板

- 掌握提示词输入与输出的结构化（角色定位、任务描述、约束条件、输出格式）
- 学会使用Few-shot示例驱动提升输出稳定性

阶段2：上下文工程与知识检索

- 理解Context vs Prompt的本质区别：Prompt是"任务入口"，Context是"智能基座"
- 掌握RAG（检索增强生成）技术，为Agent注入私域知识
- 学习如何管理短期记忆（对话上下文）与长期记忆（向量数据库）

阶段3：工具函数的系统化设计

- 设计并实现可供Agent调用的工具与API接口
- 理解Function Calling的工作原理
- 学会使用LangChain等框架的工具封装模式

阶段4：Agent规划与多Agent协作

- 掌握ReAct（Reasoning + Acting）等Agent推理模式
- 学习Agentic Workflow的四大设计模式：自我反思、工具使用、自主规划、多智能体协作
- 探索多Agent系统的编排与协调

5.2 学习路径

角色	入门（1-2周）	进阶（3-6周）	专家（第3月起）
开发者	掌握Python + LangGraph 框架	实现RAG知识库与本地模型部署	构建MAS多智能体分布式系统

5.3 Workflow 和 Agent 的差异

Workflow vs Agent

固定流程 vs 智能代理

Workflow

预定义 workflow



固定流程

按照预设步骤执行



线性执行

从开始到结束顺序执行



明确目标

每个步骤目标清晰

步骤 1



步骤 2



步骤 3



完成

局限性

- 无法适应意外情况
- 缺乏自主决策能力
- 不能学习和优化

Agent

智能代理系统



自主思考

根据情况动态决策



动态调整

灵活改变执行路径



工具使用

按需调用外部工具

接收任务



思考 & 规划



决策 & 执行

循环迭代



达成目标

优势

- 适应复杂多变的场景
- 自主判断和决策
- 持续学习和优化

核心差异

Workflow

机械执行, 适合标准化任务

Agent

智能决策, 适合复杂动态任务

5.4 多智能体系统

“多智能体系统”是人工智能领域的一个重要分支。简单来说，它研究的是如何让一组相对独立的“智能体”通过相互协作、协调或竞争，来共同完成单个智能体难以甚至无法完成的复杂任务。

六、Agent开发中的关键技术与最佳实践

6.1 Agentic Workflow四大设计模式

吴恩达提出，智能体工作流的性能往往比模型本身的规模更重要。以下是2026年主流的四
种模式³：

模式	说明	示例场景
自我反思（ Reflection ）	Agent生成结果后，自己检查错误并修正	代码生成后自动review
工具使用（ Tool Use ）	遇到不懂的问题，主动搜索或运行代码	联网获取实时信息
自主规划（ Planning ）	面对模糊目标，自动规划执行路径	"帮我规划一个旅行方案"
多智能体协作（ Multi-agent Collaboration ）	让"程序员Agent"写代码，"测试员Agent"找Bug	复杂软件开发流程

6.2 专家避坑指南

开发Agent的4个建议⁴：

- 1. 拒绝宽泛指令：避免说"帮我写篇文章"，应说"基于附件中的3个案例，写一篇800字的行业深度评析"
- 2. 避免过度依赖单次对话：复杂智能体需要多次迭代，通过Feedback Loop不断修正Prompt逻辑
- 3. 注意Token成本与幻觉：关键环节必须引入"人机协作（Human-in-the-loop）"进行审核
- 4. 从小处着手：先找出一个每天重复超过3次的枯燥任务，用简单Prompt定义角色，再逐步挂载插件

6.3 工具调用与记忆管理

工具调用：Agent最强大的能力在于能驱动外部世界。通过调用API、运行Python脚本或操作浏览器插件，它能完成"知行合一"。

规范的工具定义需要具备以下三大核心要素：

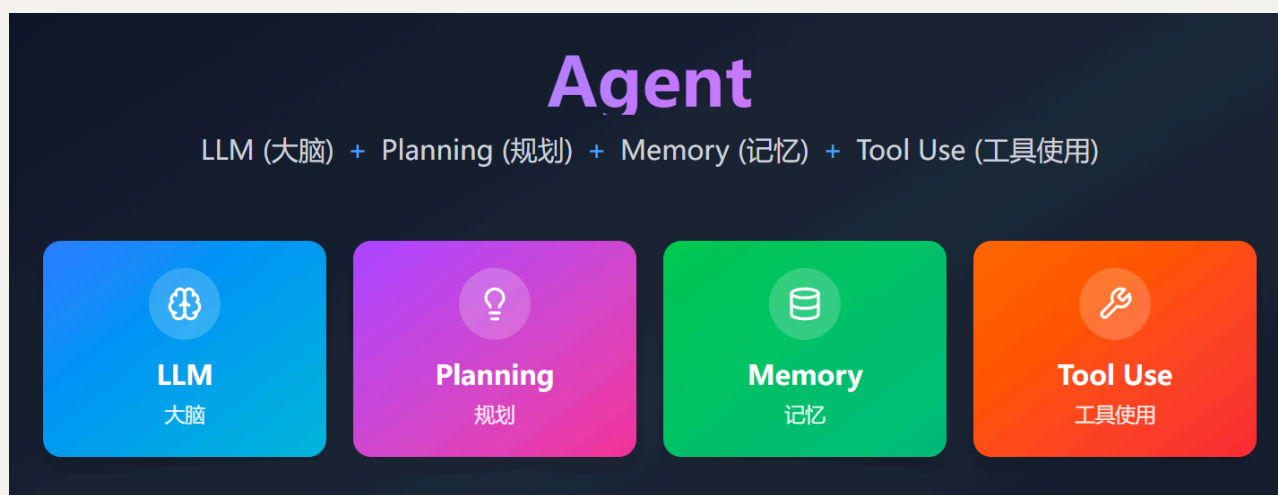
- 名称：简洁且唯一的标识符，供智能体在触发 **Action** 时准确引用（如 **Search**）。
- 描述：以自然语言精准阐明工具的功能边界与适用场景。这是工具定义的重中之重，大语言模型完全依赖该描述来进行工具的选择与路由决策。
- 执行逻辑：真正承载业务意图、执行具体任务的后端函数或方法。

记忆管理：

- 短期记忆：利用Context Window记录当前对话逻辑
- 长期记忆：通过RAG技术将海量知识存储在向量数据库中，实现跨会话的上下文贯通

七、总结与行动建议

7.1 核心公式



Agent的本质是"模型 + 结构 + 机制"的组合体，而不是模型本身。

7.2 三步入门行动指南

1. 确定场景：找一个你每天重复超过3次的枯燥任务（如整理日报、抓取资讯）作为第一个Agent项目
2. 选择工具：
 - 不想写代码 → Coze/Dify（低代码平台）
 - Python开发者 → LangChain
 - 多Agent场景 → AutoGen/CrewAI
3. 小步快跑：先用简单Prompt定义角色，再逐步添加工具、挂载知识库，持续根据反馈优化

AI Agent正在将AI从"对话框"进化为"操作系统"，掌握Agent开发能力将成为未来技术人的核心竞争力。正如一位从业者所说："未来的竞争，本质上是'编排智能'的竞争。"

参考文献：

-
1. AI时代不可不知！AI Agent 架构全解析：從 Google 白皮書到企業落地的技術藍圖. <https://ideas-dtri.iii.org.tw/%e7%94%a2%e6%a5%ad%e8%b3%87%e8%a8%8a/ai-agent-3/> ↩
 2. 零基础指南：从 Prompt 到上下文工程构建 AI Agent. <https://cloud.tencent.com.cn/developer/article/2586420?policyId=1003> ↩
 3. 2026年智能体（Agent）怎么学？从入门到实战的全景避坑指南. <https://developer.aliyun.com/article/1707471> ↩
 4. 智能体来了：真正从0-1，让AI为你所用. <https://developer.aliyun.com/article/1708908> ↩